STARVNT

Business Truth + Core + Aura+ + Universal Vendor OS + Category Intelligence + Matching + Commercial + Payment + Execution + Protection + Settlement + QA/UAT

MASTER ENGINEERING CONSTITUTION

Build for the operating system — not just the screen.

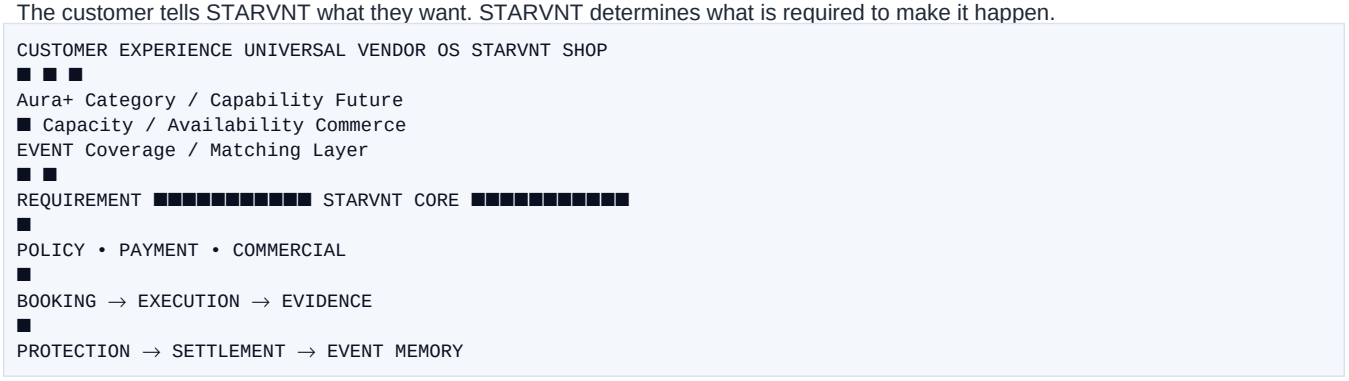
Single Source of Truth • Category Intelligence • Backend Authority • Production QA/UAT

EXECUTIVE ARCHITECTURE

```
CUSTOMER → AURA+ → EVENT → REQUIREMENT → CATEGORY INTELLIGENCE → MATCHING → VENDOR AURA+ → OPPORTUNITY → QUOTE
→ PAYMENT → BOOKING → EXECUTION → EVIDENCE → VALIDATION → PROTECTION / RESOLUTION → SETTLEMENT → EVENT MEMORY
```

CUSTOMER COMPLEXITY = LOW
INTERNAL STARVNT COMPLEXITY = HIGH

The customer tells STARVNT what they want. STARVNT determines what is required to make it happen.



0. DEVELOPER MANDATE

You are building STARVNT — an Event Operating System, not a collection of dashboards.

The platform is one connected operating loop: Customer → Aura+ → Event → Requirement → Category Intelligence → Matching → Vendor Aura+ → Opportunity → Quote → Payment → Booking → Execution → Evidence → Validation → Protection/Resolution → Settlement → Event Memory.

Core principle: CUSTOMER COMPLEXITY = LOW; INTERNAL STARVNT COMPLEXITY = HIGH.

1. NON-NEGOTIABLE ARCHITECTURAL RULES

ONE CONNECTED PLATFORM: no parallel business systems or conflicting truths.

CORE IS THE BUSINESS AUTHORITY for Customer, Organization, Event, Requirement, eligibility, availability, quote, booking, payment status, execution, completion, protection, settlement and audit.

AURA+ IS NOT THE BUSINESS AUTHORITY: it may understand, analyse, clarify, explain, compare, recommend and assist, but it cannot invent eligibility, availability, payment success, booking confirmation, settlement eligibility, deterministic commercial calculations, completion or vendor truth.

2. MASTER BUSINESS MODEL

CUSTOMER → EVENT → REQUIREMENT → SERVICE LOCATION → CATEGORY → SERVICE → CAPABILITY → CAPACITY → COVERAGE → AVAILABILITY → ELIGIBILITY → MATCHING → OPPORTUNITY → VENDOR RESPONSE → RANKING → VALIDATED OPTIONS → QUOTE → PAYMENT POLICY → PAYMENT → BOOKING → EXECUTION → EVIDENCE → COMPLETION VALIDATION → PROTECTION / RESOLUTION → SETTLEMENT → REVIEW / PERFORMANCE → EVENT MEMORY.

3. EVENT ≠ BOOKING

One Event may contain many Requirements, Vendors, Quotes, Bookings, Payments, Execution records and Completion records.

Never model ONE EVENT = ONE BOOKING.

4. REQUIREMENT ENGINE

Every customer need becomes a structured Requirement with event/category/service/date/time/duration/location/style/capability/capacity/budget/priority/status/source/confidence/audit timestamps.

Statuses: KNOWN, CONFIRMED, INFERRED, MISSING, AMBIGUOUS, RECOMMENDED.

INFERRED ≠ CONFIRMED. Aura+ must not convert inference into confirmation without user confirmation.

5. CATEGORY INTELLIGENCE ENGINE — P0

```
Universal flow: CATEGORY → SERVICES → REQUIREMENTS → CAPABILITIES → CAPACITY → RESOURCES → COVERAGE → AVAILABILITY → PRICING → MATCHING → QUOTE → BOOKING → EXECUTION → COMPLETION → SETTLEMENT.
```

Use a universal base model plus category-specific modules. Do not force identical fields across categories.

6. HARD CATEGORY SEPARATION

Caterer ≠ Decorator; Photographer ≠ Videographer; Videographer ≠ Cinematographer; DJ ≠ Sound Vendor; Singer ≠ Band; Anchor ≠ Generic Artist; Makeup Artist ≠ Hair Stylist; Florist ≠ Decorator; Drone Operator ≠ Photographer; Venue ≠ Accommodation; Planner ≠ Coordinator.

Category mismatch must prevent matching.

7. CATEGORY CONTRACT — MANDATORY

No category enters production without answers to: what is sold; services; customer inputs; capabilities; capacity; resources; location; availability; coverage; travel; pricing; matching; quoting; booking; execution; completion proof; partial failure; evidence; settlement; performance signals; Aura+ questions; customer-facing data; vendor-facing data; protected information.

8. UNIVERSAL VENDOR DATA MODEL

Identity; Category; Services; Capability; Capacity; Location/Coverage; Availability; Commercial; Portfolio; Operations; Reputation; Security/Audit.

Capacity includes team, equipment, inventory, vehicles, space, daily/concurrent/geographic capacity.

9. VENDOR ACTIVATION

```
DRAFT → REGISTERED → PROFILE_INCOMPLETE → VERIFICATION_PENDING → VERIFIED → ELIGIBLE → ACTIVE.
```

ACTIVE is not merely profile completion. Required service model must be complete: category, service, capability, capacity, coverage, availability, pricing and required verification.

10. PRIMARY CATEGORY + ADDITIONAL SERVICES

Every vendor has a Primary Category plus Additional Services / Secondary Categories.

An additional service requires its own capability, capacity, coverage, availability, pricing, verification and execution profile.

11. CATEGORY REGISTRY

Initial taxonomy spans Planning & Management; Venue & Accommodation; Decor & Production; Food; Media; Beauty; Entertainment; Fashion; Logistics; Event Infrastructure; Religious/Cultural; Event Technology.

Examples include Planner, Wedding Planner, Decorator, Florist, Stage, Lighting, Caterer, Photographer, Videographer, Cinematographer, Drone, Makeup, DJ, Singer, Band, Anchor, Bridal/Groom Wear, Transport, Security, Purohit, QR Check-in, RFID, Event App, Streaming and Projection.

Architecture must allow new categories without rewriting Core.

12. CATEGORY-SPECIFIC OPERATIONAL INTELLIGENCE

Caterer: cuisine/menu/guest capacity/meal type/serving/live counters/kitchen/staff/equipment/refill/cleanup.

Decorator: stage/mandap/floral/entrance/ceiling/lighting/draping/tablescape/theme/material/inventory/crew/setup/dismantling.

Photographer: style/coverage/team/camera/lenses/second shooter/drone/deliverables/album/reels/editing.

Videographer/Cinematographer: standard/cinematic/highlight/multi-camera/live feed/drone/audio/editing/deliverables.

DJ: performance/sound/equipment/console/speakers/line array/subwoofer/technician/lighting/power/setup/duration.

Singer/Band: genre/language/team/instruments/sets/soundcheck/travel/accommodation/rider.

Anchor/MC: language/audience/script/stage/programme/games/announcements/rehearsal/duration.

Beauty: bridal/groom/family/count/type/hairstyling/draping/touch-up/call time/team/travel.

Purohit/Ceremony: ceremony/tradition/language/duration/materials/count/location/travel.

13. CATEGORY-SPECIFIC CAPABILITY SCORE

Maintain Vendor Trust Score plus Category Capability Score.

Examples: caterer cuisine/capacity/menu/kitchen/staff/service/venue logistics; decorator design/inventory/crew/material/stage/setup; photographer coverage/team/camera/cinematography/drone/deliverables; DJ performance/sound/equipment/technician/event capacity; anchor language/audience/experience/stage/programme coordination.

14. VERIFICATION LADDER

```
SELF_DECLARED → PHONE_VERIFIED → KYC_VERIFIED → DOCUMENT_VERIFIED → PORTFOLIO_VERIFIED → BOOKING_VERIFIED → STARVNT_VERIFIED.
```

A vendor claim is not equivalent to verified execution history.

15. AVAILABILITY ENGINE

Availability considers date, time, duration, location, team, equipment, inventory, vehicle, space, existing bookings, holds, travel, setup, buffer and concurrent-job limits.

Vendor available ≠ Resource available.

16. BOOKING CAPACITY RESERVATION

Confirmed bookings reserve applicable calendar, team, equipment, inventory, vehicle, space and concurrent-job capacity. Prevent double-booking.

17. REAL-TIME OPPORTUNITY ENGINE — P0

```
Customer → Aura+ → Requirement → Core → Category Engine → Eligibility → Availability → Capacity → Travel → Validated Vendor Pool → Notification Engine → Vendor Aura+.
```

Never notify vendors before eligibility.

18. GEO MATCHING

Default initial radius = 25 KM. It must be a Policy Engine configuration, not a universal hard-coded rule. Category/service-specific radius policies are required.

19. SEARCH WAVES

Wave 1: initial policy radius. Wave 2: approved expanded radius. Wave 3: specialist/long-distance search where permitted.

Never silently expand geography; Aura+ explains expansion.

20. 60-SECOND VENDOR OPPORTUNITY

60 seconds is a response SLA, not a winner-selection rule.

Responses: CAN_EXECUTE, NEED_CLARIFICATION, NOT_AVAILABLE. Fastest response does not automatically win.

21. ELIGIBILITY ≠ RANKING

Hard filter first; eligible pool second; ranking third.

Ranking may consider scope fit, availability, capacity, capability, validated total cost, travel, reliability, style/service fit, verification, historical performance, response behaviour and customer fit.

22. CUSTOMER OPTIONS

Maximum 4 validated options. Show 0, 1, 2, 3 or 4 based on actual qualified vendors. Never manufacture a fourth option. Backend determines the top four; frontend does not.

23. CUSTOMER AURA+

Natural language + voice; intent; requirement building; meaningful questions; missing information; option explanation; comparison; selection rationale; risk; budget impact; next action.

Aura+ explains the ranking; Core produces the ranking.

24. VENDOR AURA+

Job Interpreter; Visual/Scope Brief; Quote Assistant; Execution Assistant.

May summarize scope, analyse references, identify missing data and help with quotes/execution plans. Must not fabricate availability, price, booking, payment, approval, completion or settlement.

25. PROGRESSIVE INFORMATION DISCLOSURE

Before booking: general location, date, event type, quantity, scope and permitted commercial context. After booking: more operational location detail. Execution stage: exact access, contact and arrival information.

Enforce at API/backend level, not merely in UI.

26. COMMERCIAL POLICY ENGINE — P0

Policy-driven platform/category/service/payment fees, tax, travel, logistics, promotion, vendor settlement, cancellation, refund, reservation amount and settlement delay.

Business share changes must not require rewriting Core booking architecture.

27. RESERVATION POLICY

Do not globally hard-code 30% advance. Payment Policy Engine supports PERCENTAGE, FIXED, MILESTONE, SERVICE_SPECIFIC, CATEGORY_SPECIFIC and CUSTOM POLICY.

28. QUOTE ENGINE

Version every quote. Store quote ID/version/scope/price/taxes/fees/travel/inclusions/exclusions/valid-until/cancellation/reservation policy/vendor/requirement reference.

Enforce expiry; old versions cannot silently become current.

29. PAYMENT ARCHITECTURE

```
QUOTE → APPROVAL → PAYMENT POLICY → PAYMENT INTENT → GATEWAY → WEBHOOK → SERVER VERIFICATION → PAYMENT RECORD → BOOKING CONFIRMATION.
```

Payment must be server-verified, idempotent, audited and reconciled. Never trust frontend success.

30. PAYMENT ≠ SETTLEMENT

Customer payment = money received/verified. Vendor settlement = earned after applicable execution/completion validation.

31. EXECUTION ENGINE

Execution is category-aware. Each category owns steps, responsible party, timing, resources, milestones, evidence and completion criteria.

32. COMPLETION VALIDATION

```
SERVICE_COMPLETED → COMPLETION_EVIDENCE → CORE_VALIDATION → COMPLETION_VERIFIED → SETTLEMENT_ELIGIBILITY → SETTLEMENT.
```

Outcomes: PASS, REVIEW, FAIL.

33. PARTIAL SERVICE FAILURE

Support PASS, PARTIAL, REVIEW, FAIL. Example: 8-hour photography booked, 5 hours delivered may trigger partial refund, partial settlement, additional deliverable, recovery, replacement or other resolution.

34. CATEGORY-SPECIFIC COMPLETION EVIDENCE

Caterer: service/meals/guest count/service window/customer confirmation. Photographer: coverage/deliverables/files/album. Decorator: setup/execution/dismantling. Singer: performance/set duration. Anchor: programme/stage coordination/duration. Purohit: ceremony/agreed ritual scope.

35. CUSTOMER PROTECTION ENGINE — P0

Protection is a business system, not a generic support ticket. Case links customer, event, requirement, booking, vendor, payment, completion, evidence, issue, policy, resolution and audit.

```
Lifecycle: NONE → AT_RISK → PROTECTED → ISSUE_REPORTED → UNDER_REVIEW → RESOLUTION_PROPOSED → RESOLVED.
```

36. VENDOR CANCELLATION / REPLACEMENT

```
VENDOR_CANCELLED → BOOKING_AT_RISK → CUSTOMER_PROTECTION → ALTERNATIVE_DISCOVERY → VALIDATION →
CUSTOMER_DECISION → QUOTE → COMMERCIAL ADJUSTMENT → PAYMENT IF REQUIRED → REBOOKING.
```

Never end with “Vendor cancelled.”

37. FAILURE MATRIX

Mandatory failure handling includes payment, duplicate webhook/booking/event/notification, vendor cancellation, customer issue, incomplete/partial service, payment mismatch, invalid state, unauthorized access, automation/matching/notification failure, quote expiry, settlement exception, protection and replacement failure.

Every failure needs detection, state, recovery, audit, user communication and final resolution.

38. AUDIT ENGINE — P0

Every critical state change records WHO, WHAT, WHEN, FROM_STATE, TO_STATE, WHY, SOURCE, AUTHORITY, REFERENCE_ID and IDEMPOTENCY_KEY.

The system must answer why a booking was confirmed.

39. SECURITY / OWNERSHIP

Backend authorization is mandatory. Vendor A cannot access Vendor B leads, quotes, customers, bookings, payments, private documents, commercial data or messages. Customer A cannot access Customer B.

Frontend filtering is not security.

40. MESSAGING CONTEXT

```
EVENT → REQUIREMENT → VENDOR → QUOTE / BOOKING → MESSAGE.
```

Do not build a generic WhatsApp clone disconnected from business state.

41. EVENT CIRCLE

May include customer, authorized family, authorized participants and booked vendors. Access is controlled by role and event membership.

42. EVENT DAY COMMAND CENTER

Live event surface: vendors, arrival, setup, services, timeline, payments, issues, customer requests, emergency and completion.

Aura+ can surface live facts only when those facts originate in Core/operational systems.

43. VENDOR PERFORMANCE INTELLIGENCE

Track response rate/time, quote turnaround, acceptance, cancellation, on-time arrival, completion, issue/resolution rate, satisfaction, verified execution history and category-specific performance.

44. RANKING ENGINE

```
Deterministic, explainable pipeline: Hard Filter → Eligibility → Availability → Capacity → Coverage → Travel → Capability → Commercial Fit → Reliability → Historical Performance → Customer Fit → Ranking → Top Validated Options. Aura+ explains; Core decides.
```

45. BUDGET INTELLIGENCE

```
Budget → Committed Cost → Pending Cost → Validated Cost → Estimated Cost → Potential Additional Cost → Budget Risk → Alternative → Recommendation. Calculations must use validated system data.
```

46. ATTENTION ENGINE

Do not spam. Surface only meaningful, event-aware, state-aware actions such as quote expiry, decision required, confirmation and payment verification.

47. EVENT TIMELINE

```
Event Created → Requirement Added → Vendor Matched → Quote Received → Quote Approved → Payment Verified → Booking Confirmed → Vendor Coordination → Event Day → Execution → Completion → Settlement → Event Complete → Memory.
```

48. EVENT MEMORY

Completed events become historical context. May include photos, videos, vendors, experiences, preferences, milestones, event story and verified outcomes. Privacy and consent are mandatory.

49. STARVNT SHOP — COMING SOON

Visible future commerce layer: “Everything your celebration needs, coming soon.”

Current phase = placeholder only unless separately approved. Future categories include wedding essentials, décor products, bridal/groom accessories, gifts/hampers, invitations, event merchandise, photography accessories, fashion/jewellery, lifestyle and STARVNT collections.

```
Future architecture: PRODUCT → CATEGORY → VARIANT → PRICE → INVENTORY → CART → ORDER → PAYMENT → FULFILMENT → DELIVERY → RETURN / REFUND.
```

PRODUCT ORDER ≠ EVENT SERVICE BOOKING.

50. UNIFIED CUSTOMER IDENTITY

One STARVNT account eventually contains Events, Requirements, Vendor Bookings, Payments, Orders, Wishlist, Messages, Event Memory and Shop.

51. ADMIN CONTROL TOWER

One operational control layer for live events, requirements, matching, vendors, quotes, bookings, payments, protection, settlements, orders, system health, automation/notification failures and audit. Admin is not a second source of truth.

52. OBSERVABILITY

Expose API, database and queue health; matching jobs; notification delivery; payment webhooks; retries; automation/AI/integration failures; latency; audit failures and error rates.

53. IDEMPOTENCY

Mandatory for payment webhook, booking/event/requirement creation, opportunity notification, quote submission, payment confirmation, settlement, refund, replacement and notifications. Duplicate requests must not create duplicate business objects.

54. FEATURE FLAGS / SAFE MODE

Feature flags for matching, Aura+ recommendations, notifications, payment, replacement and automated settlement. Emergency modes: Matching Safe Mode; Aura+ Recommendation Human Review; Automation Disabled.

55. HUMAN-IN-THE-LOOP

```
High-risk actions may require review: large-value booking, refund, dispute, protection, replacement, suspicious vendor, payment mismatch, settlement exception. Flow: AURA+ → DETECT → CORE → HUMAN REVIEW → DECISION → AUDIT.
```

56. DEVELOPER EXECUTION ORDER

P0 Business Truth: Core State, Event, Requirement, Category Engine, Vendor Capability, Capacity, Coverage, Availability, Travel, Matching, 60-sec Opportunity, Commercial Policy, Quote, Payment, Booking, Execution, Completion, Protection, Settlement, Audit, Security, Verification.

P1 Intelligence/Operations: Customer Aura+, Vendor Aura+, Ranking, Replacement, Performance, Budget Intelligence, Risk, Attention, Timeline, Event Day Command Center.

P2 Experience: Event Circle, Event Memory, Advanced Analytics, Advanced Aura+ UX, Mobile polish, Accessibility, Personalization.

Future: Shop full Commerce, Multi-city expansion, Advanced marketplace intelligence.

57. KOLKATA-FIRST SCALE STRATEGY

Do not immediately expand every category across India. Build category depth first.

```
Initial liquidity focus: Wedding Planning, Decoration, Photography. Then Catering → Makeup → DJ/Sound → Anchor → Singer/Band → Transport → Venue → Entertainment → Other categories.
```

58. DEVELOPMENT RULE — REUSE → INTEGRATE → TEST → PROVE

Before new code: search existing implementation; reuse models/APIs/components; integrate with Core; test; prove.

Do not duplicate event, vendor, booking, payment, customer, matching, notification or availability logic.

59. EVERY FEATURE REQUIRES 5 PROOFS

1 UI Proof; 2 API Proof; 3 Database Proof; 4 Business Logic Proof; 5 E2E Proof.

```
Example payment proof: Gateway → Webhook → Server Verification → Payment DB → Core Verification → Booking DB → Audit → Vendor UI → Customer UI.
```

60. QA / UAT RULE

A module is not complete because UI looks finished, API returns 200, database has a row or developer says “done”.

Completion requires Business Truth + Category Correctness + Security + Idempotency + Auditability + E2E Behaviour.

61. MANDATORY P0 UAT

Category: correct category matched; wrong category excluded; category fields/capabilities enforced.

Availability: unavailable vendor/resource/team/equipment conflicts excluded.

Geo: policy, override, travel, search waves work; silent expansion impossible.

Opportunity: only eligible vendors notified; 60-second window; refresh does not restart; duplicates safe.

Ranking: fastest does not automatically win; deterministic; top 4 backend-generated; fewer than 4 stays fewer.

Quote: versioning and expiry. Payment: frontend cannot fake success; duplicate webhook safe; payment ≠ settlement.

Booking: policy respected; capacity reserved; duplicates prevented. Execution: category workflow; evidence; partial completion. Protection: cancellation triggers protection; replacement works; issue creates case. Security: cross-vendor/customer blocked; API authorization. Audit: critical transitions logged with actor/source/reference/idempotency.

62. FINAL ARCHITECTURE

STARVNT = Customer Experience + Universal Vendor OS + Future Shop, all connected to STARVNT Core.

```
Core feeds Policy, Payment and Commercial layers, then Booking → Execution → Evidence → Completion Validation → Protection → Resolution → Settlement → Event Memory.
```

63. FINAL ENGINEERING CONSTITUTION

1. One Event, Many Requirements. 2. One Requirement, One Authoritative Business Truth. 3. Every Category Shares One Core but Owns Category-Specific Operational Intelligence. 4. Service ≠ Capability ≠ Resource. 5. Vendor Availability ≠ Resource Availability. 6. Eligibility ≠ Ranking. 7. 60 seconds is a response SLA, not a winner rule. 8. 25 km is a policy default, not a hard-coded universal rule. 9. Up to four validated options — never four artificial options. 10. Payment ≠ Booking ≠ Settlement. 11. Customer payment is verified before booking confirmation. 12. Vendor settlement is earned through successful delivery and validation. 13. Completion is category-specific. 14. Protection is a business system. 15. Aura+ may understand/analyse/recommend/explain/assist — Core decides truth. 16. Frontend cannot manufacture truth. 17. UI completion is not system completion. 18. Critical state transitions are auditable. 19. Product Order ≠ Event Service Booking. 20. Shop is a future Commerce Layer, not a parallel Event Core.

64. FINAL DEVELOPER ACCEPTANCE TEST

Do not report “feature complete” unless all applicable answers are YES: business truth in Core; ownership enforced; category/service correct; capabilities correct; capacity sufficient; coverage valid; availability real; travel feasible; eligibility deterministic; matching backend-authoritative; notifications only to eligible vendors; 60-second window enforced; ranking deterministic; options validated; Aura+ explains rather than invents; commercial and reservation policies are driven by policy; payment server-verified/idempotent; booking atomic; capacity reserved; execution category-aware; completion evidence captured/validated; protection/replacement available; settlement separated from payment; critical state audited; cross-vendor access blocked; complete E2E flow proven.

If any answer is NO: STATUS = NOT COMPLETE.

FINAL PRODUCT PRINCIPLE

STARVNT is not a vendor directory, wedding CRM, generic marketplace, chatbot, social-media app or collection of dashboards.

STARVNT is AN EVENT OPERATING SYSTEM.

Moat = Aura+ + Category Intelligence + Real-Time Availability + Vendor Capability Graph + Matching + Core Business Truth + Payment Trust + Execution Data + Customer Protection + Event Memory.

The customer tells STARVNT what they want. STARVNT determines what is required. Core determines what is true. Matching determines who can execute. Aura+ makes complexity understandable. Vendors execute. Evidence proves outcome. Protection handles failure. Settlement follows validated delivery. Event Memory creates the next relationship.

BUILD FOR THAT. DO NOT BUILD JUST FOR THE SCREEN.